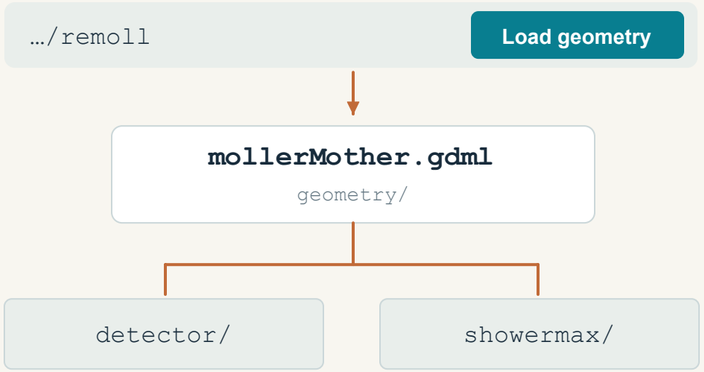
Loading

1 Read GDML

PYTHON → C++ / GEANT4



Referenced files → Geant4 volumes

run.py:72–78

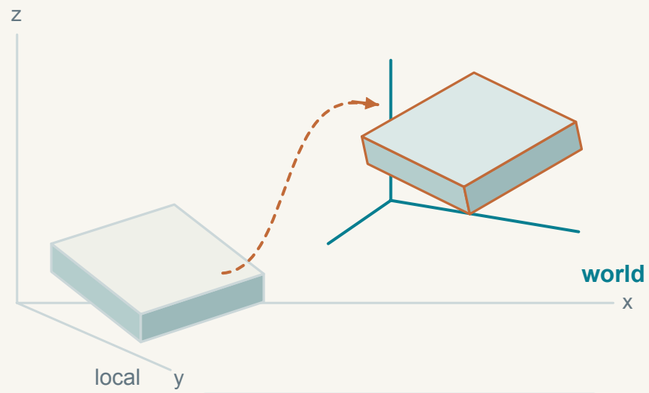
```
78  execute([executable, gdml, OUTPUT /
    'geometry.next.json'], log)
```

export_geometry.cc:98–104

```
100 parser.Read(gdml.filename().string(), false);
```

2 Place volumes

C++ / GEANT4



parent × child placement

export_geometry.cc:44–45, 62, 85

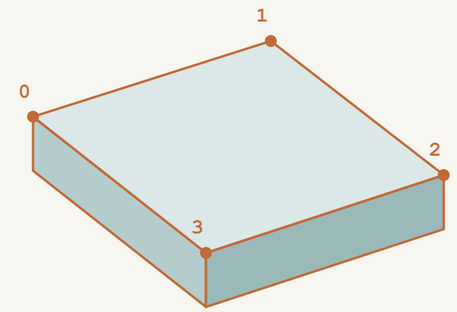
```
62  const auto world = rotation * local + position;
```

export_geometry.cc:64–65

```
64  const auto check =
    history.GetTopTransform().Inverse().TransformPoint(local);
```

3 Extract surfaces

C++ / GEANT4



vertices `[x, y, z]`

faces `[0, 1, 2, 3]`

World coordinates · mm

export_geometry.cc:50, 59–76

```
50  auto* mesh = volume->GetSolid()->GetPolyhedron();
    …
72  int count, indices[4];
73  mesh->GetFacet(i, count, indices);
```

4 Send geometry

C++ → PYTHON → BROWSER

geometry.json

```
{
"meshes": [{
"name": …,
"system": …,
"geometry_detector": …,
"vertices": [[x,y,z], …],
"faces": [[0,1,2,3], …]
}]
}
```

/status → /geometry

run.py:79–83, 123–128

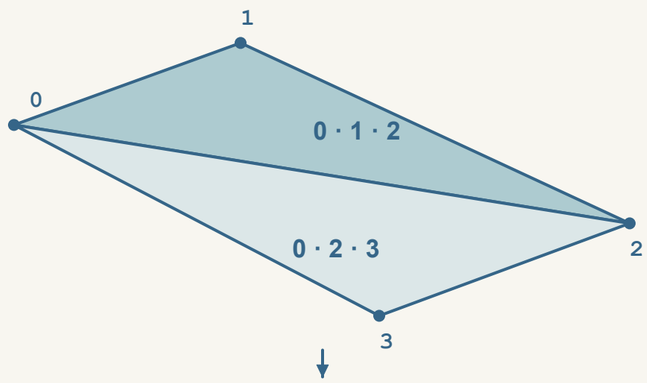
```
80  (OUTPUT / 'geometry.next.json').replace(OUTPUT /
    'geometry.json')
```

view.js:122–130

```
127 const geometry=await api('/geometry');
```

5 Build the view

JAVASCRIPT / WEBGL



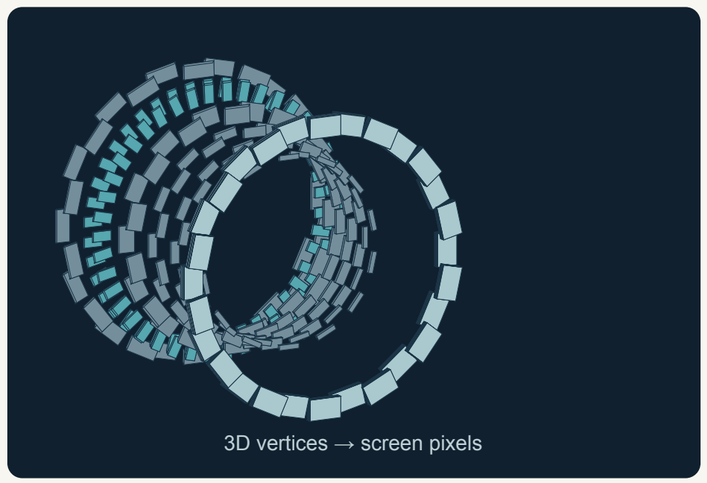
Triangle vertices → GPU buffer

view.js:51–66

```
61  for(let i=1;i+1<points.length;i++) for(const p of
    [points[0],points[i],points[i+1]])
    surfaces.push(...position(p),...color);
    …
66  gl.bufferData(gl.ARRAY_BUFFER,new
    Float32Array([...surfaces,...edges]),gl.STATIC_DRAW);
```

6 Draw the detector

WEBGL



view.js:19–24, 70–80

```
23  gl_Position=vec4(12.*s*p.x/screen.x+2.*pan.x/screen.x*d,
24  12.*s*p.y/screen.y-2.*pan.y/screen.y*d,1.004008*d-.2004008,d);
    …
79  gl.drawArrays(gl.TRIANGLES,0,triangles);
    gl.disable(gl.POLYGON_OFFSET_FILL);
80  gl.drawArrays(gl.LINES,triangles,lines);
```